

# Building a Movie Recommendation System

## Introduction

In the age of digital streaming platforms, the sheer volume of available movies and TV shows can be overwhelming. With thousands of options at our fingertips, how do we decide what to watch next? This is where **movie recommendation systems** come into play. These systems help users discover new content based on their preferences, making the decision-making process easier and more enjoyable.

In this, we will explore the process of building a **movie recommendation system** from scratch. We will walk through the steps involved in creating a system that suggests movies based on their similarity to a given movie. This project is a perfect blend of data science, machine learning, and software engineering, and it provides valuable insights into how recommendation systems work in the real world.

## The Dataset

The foundation of any recommendation system is **data**. For this project, we used two datasets:

1. **tmdb\_5000\_movies.csv**: This dataset contains information about 5000 movies, including details like the movie's title, budget, genres, keywords, overview, popularity, production companies, release date, revenue, runtime, and more.
2. **tmdb\_5000\_credits.csv**: This dataset contains information about the cast and crew of the same 5000 movies. It includes details like the movie's title, cast members, and crew members.

These datasets were sourced from **The Movie Database (TMDb)**, a popular online database for movies and TV shows. By combining these two datasets, we were able to create a comprehensive dataset that includes both the metadata of the movies and the people involved in their production.

---

## Data Preprocessing

Before we could build the recommendation system, we needed to clean and preprocess the data. This step is crucial because raw data often contains inconsistencies, missing values, and irrelevant information that can affect the performance of the system.

### 1. Merging the Datasets

- The first step was to merge the two datasets (tmdb\_5000\_movies.csv and tmdb\_5000\_credits.csv) based on the movie title. This allowed us to combine the movie metadata with the cast and crew information.

### 2. Selecting Relevant Features

- Not all features in the dataset were relevant for our recommendation system. We selected only the most important features, such as:
  - **Movie ID**: A unique identifier for each movie.
  - **Genres**: The genres associated with the movie (e.g., Action, Comedy, Drama).

- **Keywords:** Keywords that describe the movie's themes or plot points.
- **Cast:** The main actors in the movie.
- **Crew:** The key crew members, particularly the director.
- **Title:** The title of the movie.
- **Overview:** A brief summary of the movie's plot.

### 3. Handling Missing Values

- Some movies had missing values in the "overview" column. Since the overview is a critical part of the recommendation system, we removed movies with missing overviews.

### 4. Data Transformation

- The data in the "genres," "keywords," "cast," and "crew" columns were stored in a JSON-like format. We parsed these columns to extract relevant information. For example, from the "genres" column, we extracted the names of the genres associated with each movie.
- We also limited the number of cast members to the top 3 actors for each movie to reduce noise in the data.

### 5. Text Preprocessing

- To prepare the text data for analysis, we performed several preprocessing steps:
  - **Lowercasing:** Converted all text to lowercase to ensure consistency.
  - **Stemming:** Reduced words to their root form (e.g., "running" to "run") using the Porter Stemmer algorithm.
  - **Removing Spaces:** Removed spaces between words to create single tokens (e.g., "science fiction" became "sciencefiction").

### 6. Creating Tags

- We combined the "overview," "genres," "keywords," "cast," and "crew" columns into a single "tag" column. This tag column served as the input for our recommendation system, as it contained all the relevant information about each movie in a single text string.

---

## Building the Recommendation System

With the pre-processed data in hand, we moved on to building the recommendation system. The core idea behind our system is to recommend movies that are similar to a given movie based on their tags.

### 1. Vectorization

- To compare the tags of different movies, we needed to convert the text data into numerical form. We used the **CountVectorizer** from the sklearn library to convert the tags into a matrix of token counts. This process is known as **vectorization**.
- The CountVectorizer also removed common English stopwords (e.g., "the," "and," "is") to reduce noise in the data.

## 2. Cosine Similarity

- Once the tags were vectorized, we calculated the **cosine similarity** between all pairs of movies. Cosine similarity is a metric used to measure how similar two vectors are, regardless of their size. In this context, it measures how similar the tags of two movies are.
- The result was a **similarity matrix**, where each cell represented the similarity score between two movies. A score of 1 indicates that the movies are identical, while a score of 0 indicates no similarity.

## 3. Recommendation Algorithm

- To recommend movies, we implemented a simple algorithm:
  1. Given a movie title, find its index in the dataset.
  2. Retrieve the similarity scores for that movie from the similarity matrix.
  3. Sort the scores in descending order to find the most similar movies.
  4. Return the top 5 most similar movies (excluding the movie itself).

---

## Testing the System

To test the recommendation system, we selected a few popular movies and checked the recommendations generated by the system. For example:

- **Input Movie:** *Avatar*

### **Recommended Movies:**

- *Aliens vs Predator: Requiem*
- *Aliens*
- *Silent Movie*
- *Independence Day*
- *Titan A.E.*

The recommendations were based on the similarity of the tags, which included genres, keywords, cast, and crew. While some recommendations were spot-on, others were less obvious, highlighting the importance of fine-tuning the system further.

## Conclusion

Building a movie recommendation system is a fascinating journey that combines data science, machine learning, and software engineering. By leveraging the power of data, we can create systems that help users discover new content and enhance their viewing experience. While our system is a basic implementation, it lays the groundwork for more advanced recommendation systems that could revolutionize how we discover and consume media.